JWT Authentication System

Full Stack Project Documentation

Node.js + Express | Next.js 14 | TypeScript | Tailwind CSS

Backend	Frontend	Auth	Storage
Node.js + Express	**Next.js 14**	**JWT + Cookies**	**In-Memory**

1. Project Overview

This project implements a complete, production-ready JWT authentication system using a **Node.js/Express backend** and a **Next.js 14 frontend**. It demonstrates the full authentication lifecycle including registration, login, protected routes, automatic token refresh, and secure logout.

1.1 What This Project Does

- **Registers users** — hashes passwords with bcrypt, stores user data, issues tokens
- **Logs users in** — validates credentials, returns a short-lived access token + sets a long-lived httpOnly refresh token cookie
- **Protects routes** — middleware validates the Bearer access token on every protected API call
- **Auto-refreshes tokens** — the frontend silently gets a new access token when the old one expires, with zero user disruption
- **Securely logs out** — revokes the refresh token from the server store and clears the cookie

1.2 Tech Stack

Layer	Technology
Backend Runtime	Node.js with CommonJS modules
Backend Framework	Express.js v5
Authentication	jsonwebtoken (JWT)
Password Hashing	bcryptjs (cost factor 12)
Frontend Framework	Next.js 14 (App Router)
Frontend Language	TypeScript
Styling	Tailwind CSS
HTTP Client	Axios with interceptors
Security Headers	helmet, cors, express-rate-limit
Cookie Handling	cookie-parser

2. Project Structure

2.1 Backend

```
backend/
  server.js                  Main Express app entry point
  .env                       Environment variables (secrets)
  controllers/
    authController.js        register, login, refresh, logout, me
    store.js                 In-memory users Map + refreshTokens Set
  middleware/
    authMiddleware.js        Protect routes (verifies Bearer token)
    tokenUtils.js           generate/verify access & refresh tokens
  routes/
    authRoutes.js           POST /register /login /refresh /logout, GET /me
    userRoutes.js           GET /profile (protected example)
```

2.2 Frontend

```
frontend/src/
  middleware.ts              Next.js route guard (runs on Edge)
  app/
    layout.tsx              Root layout wrapping all pages with AuthProvider
    page.tsx                Root redirect to /dashboard
    globals.css             Tailwind base styles
    login/page.tsx          Login form page
    register/page.tsx       Registration form page
    dashboard/page.tsx      Protected dashboard page
  components/
    AuthShell.tsx           Reusable card wrapper for auth pages
    Input.tsx               Reusable input component with error state
  context/
    AuthContext.tsx         Global auth state (user, loading, login, logout)
  lib/
    apiClient.ts            Axios instance with auto-refresh interceptor
    auth.ts                 Auth helper functions (loginUser, registerUser...)
```

3. Core Concepts

3.1 JSON Web Tokens (JWT)

A JWT is a compact, self-contained token that encodes information (called claims) as a JSON object. It has three parts separated by dots:

```
header.payload.signature

Example decoded payload:
{
  "id": "1234567890",
  "email": "user@example.com",
  "name": "John Doe",
  "iat": 1700000000,   // issued at (Unix timestamp)
  "exp": 1700000900    // expires at (15 minutes later)
}
```

The server signs the token with a secret key. When the token arrives on a request, the server re-verifies the signature — if it matches, the token is trusted without any database lookup.

3.2 Two-Token Strategy

This project uses two different tokens with very different lifetimes and storage locations:

	Access Token
Lifetime	15 minutes
Storage	JavaScript memory
Sent via	Authorization: Bearer header
Purpose	Authenticate API requests
XSS risk	Low (memory only)
CSRF risk	None (not a cookie)

> **Why two tokens?** Short-lived access tokens limit damage if stolen. Long-lived refresh tokens are stored in httpOnly cookies that JavaScript cannot read, protecting them from XSS attacks.

3.3 httpOnly Cookies

The refresh token is stored as an httpOnly, secure, sameSite=strict cookie. This means:

- **httpOnly** — JavaScript in the browser cannot access it at all (document.cookie will not show it)
- **secure** — only sent over HTTPS in production
- **sameSite: strict** — only sent when the request originates from the same site, blocking CSRF attacks
- **path: /** — sent with all requests to the domain

3.4 Refresh Token Rotation

Every time a refresh token is used to get a new access token, the old refresh token is immediately invalidated and a new one is issued. This means:

- A stolen refresh token can only be used once before becoming invalid
- The server can detect replay attacks (same token used twice)
- Tokens are stored in a Set on the server and checked before use

4. Authentication Flows

4.1 Registration Flow

```
User fills form  →  POST /api/auth/register
                    { name, email, password }

Backend:
  1. Validate input (required fields, password length)
  2. Check email not already registered
  3. Hash password with bcrypt (cost factor 12)
  4. Save user to store: { id, name, email, hashedPassword }
  5. Generate accessToken (15m) signed with ACCESS_TOKEN_SECRET
  6. Generate refreshToken (7d) signed with REFRESH_TOKEN_SECRET
  7. Store refreshToken in refreshTokens Set
  8. Set refreshToken as httpOnly cookie
  9. Return { accessToken, user: { id, name, email } }

Frontend:
  1. Receives accessToken → stores in memory (_accessToken variable)
  2. Receives user → stores in AuthContext state
  3. Router pushes to /dashboard
```

4.2 Login Flow

```
User fills form  →  POST /api/auth/login
                    { email, password }

Backend:
  1. Validate input
  2. Look up user by email in store
  3. Compare password with bcrypt.compare()
  4. If match: generate both tokens, same as registration steps 5-9
  5. If no match: return 401 "Invalid email or password"

Frontend: same as registration steps 1-3
```

4.3 Authenticated Request Flow

```
Frontend makes any API call (e.g. GET /api/auth/me)

apiClient.ts request interceptor runs:
  → Reads _accessToken from memory
  → Adds header: Authorization: Bearer <accessToken>

Backend authMiddleware.js:
  → Reads Authorization header
  → Calls jwt.verify(token, ACCESS_TOKEN_SECRET)
  → If valid: sets req.user = decoded payload, calls next()
  → If expired: returns 401 { error: "Token expired" }
```

```
  → If invalid: returns 401 { error: "Invalid token" }
```

4.4 Auto Token Refresh Flow (The Key Feature)

This happens completely automatically — the user sees nothing.

```
1. Frontend calls GET /api/auth/me
   → Access token has expired
   → Backend returns 401

2. apiClient.ts response interceptor catches the 401:
   → Checks: is this the first retry? (original._retry flag)
   → Sets isRefreshing = true
   → Any other requests that fail while refreshing are queued

3. Interceptor calls POST /api/auth/refresh
   → Browser automatically sends httpOnly refreshToken cookie

4. Backend refresh endpoint:
   → Reads refreshToken from cookie
   → Checks it exists in refreshTokens Set
   → Calls jwt.verify(token, REFRESH_TOKEN_SECRET)
   → Generates NEW accessToken
   → Rotates refreshToken (deletes old, creates new, sets new cookie)
   → Returns { accessToken, user }

5. Interceptor receives new accessToken:
   → Calls setAccessToken(newToken) to update memory
   → Processes the queue of waiting requests with new token
   → Retries original request with new Authorization header

6. Original request succeeds
   → User never knew anything happened
```

> **Queue mechanism:** If 5 requests fail at the same time with 401, only ONE refresh call is made. The other 4 are queued and retried together once the new token arrives.

4.5 Session Restore on Page Refresh

```
User refreshes browser / opens app

1. AuthContext mounts, loading = true
2. Calls refreshSession() → POST /api/auth/refresh
   → httpOnly cookie is sent automatically by browser
3. If cookie valid: sets user in state, loading = false
4. If no cookie / expired: user = null, loading = false

5. Dashboard useEffect watches [user, loading]:
   → If loading still true: shows spinner (waits)
```

```
   → If loading false + user exists: renders dashboard
   → If loading false + no user: redirects to /login
```

4.6 Logout Flow

```
User clicks Logout  →  POST /api/auth/logout

Backend:
  1. Reads refreshToken from cookie
  2. Deletes it from refreshTokens Set (revokes it)
  3. Calls res.clearCookie("refreshToken")
  4. Returns { message: "Logged out successfully" }

Frontend:
  1. Calls clearAccessToken() → sets _accessToken = null
  2. Calls setUser(null) in AuthContext
  3. Router pushes to /login

Result: both tokens are now invalid. Even if someone had
stolen the old access token, it will expire in max 15 minutes.
```

5. API Reference

5.1 Auth Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Create new user account
POST	/api/auth/login	No	Login with email + password
POST	/api/auth/refresh	Cookie	Get new access token
POST	/api/auth/logout	Cookie	Revoke session
GET	/api/auth/me	Bearer Token	Get current user info
GET	/api/user/profile	Bearer Token	Get user profile
GET	/api/health	No	Server health check

5.2 Request / Response Examples

POST /api/auth/register

Request body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "mypassword"
}
```

Response 201:

```
{
  "message": "Account created successfully",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "1700000000000", "name": "John Doe", "email": "john@example.com"
}
}
+ Set-Cookie: refreshToken=eyJ...; HttpOnly; Path=/; SameSite=Strict
```

POST /api/auth/refresh

Request: No body needed. The httpOnly cookie is sent automatically.

Response 200:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "...", "name": "John Doe", "email": "john@example.com" }
}
```

```
+ Rotated Set-Cookie: refreshToken=eyJ...(new token)
```

6. Security Considerations

6.1 What Is Protected

Attack	Protection
XSS stealing tokens	Access token in memory only; refresh token in httpOnly cookie (JS cannot read)
CSRF attacks	sameSite=strict on cookie; access token sent via header (not cookie)
Brute force login	express-rate-limit: max 100 requests per 15 minutes per IP
Token replay	Refresh token rotation: each token can only be used once
Password exposure	bcrypt with cost factor 12 (one-way hash, never stored plain)
Header injection	helmet sets 11 security headers on every response
CORS violations	cors configured with explicit origin whitelist

6.2 Environment Variables

These secrets must never be committed to version control. Set them in your `.env` file:

```
ACCESS_TOKEN_SECRET=your_secret_min_32_chars
REFRESH_TOKEN_SECRET=another_secret_min_32_chars
ACCESS_TOKEN_EXPIRES_IN=15m
REFRESH_TOKEN_EXPIRES_IN=7d
NODE_ENV=development
CLIENT_URL=http://localhost:3000
```

> **Important:** Use two DIFFERENT secrets for access and refresh tokens. If one is compromised, the other remains secure.

6.3 Current Limitations (In-Memory Store)

- All users and tokens are lost when the server restarts
- Cannot run multiple backend instances (tokens are not shared between processes)
- No token blacklist persistence (revoked tokens come back after restart)

→ Replace store.js with MongoDB or PostgreSQL when moving to production.

7. Frontend Architecture

7.1 Token Storage Strategy

The access token is stored as a plain JavaScript variable in `apiClient.ts`:

```
// lib/apiClient.ts
let _accessToken: string | null = null;

export const setAccessToken = (token: string) => { _accessToken = token; };
export const getAccessToken = () => _accessToken;
export const clearAccessToken = () => {  accessToken = null; };
```

This is intentionally **NOT stored in localStorage or sessionStorage**. Memory storage means:

- XSS scripts cannot steal the token (they cannot access closure variables)
- Token is lost on tab close/refresh — which is fine because the refresh cookie restores it

7.2 AuthContext

The `AuthContext` provides global auth state to the entire app:

```
interface AuthContextType {
  user: User | null;      // null = not logged in
  loading: boolean;       // true while checking session on mount
  login: (email, password) => Promise<void>;
  register: (name, email, password) => Promise<void>;
  logout: () => Promise<void>;
}
```

On mount, AuthContext calls `refreshSession()` to restore the user session using the httpOnly cookie. The `loading` state is critical — it prevents the dashboard from redirecting to login while the restore is in progress.

7.3 Axios Interceptors

Request interceptor — runs before every request:

```
api.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Response interceptor — runs when any request fails:

```
api.interceptors.response.use(
  (response) => response,         // success: pass through
  async (error) => {
```

```
    if (error.response?.status !== 401 || original._retry) {
      return Promise.reject(error);  // not a token error, reject normally
    }
    original._retry = true;
    // ... call /auth/refresh, update token, retry original request
  }
);
```

7.4 Route Protection

Two layers of protection:

Layer 1 — Next.js Middleware (src/middleware.ts)

Runs on the Edge before the page renders. Checks for the `refreshToken` cookie:

- No cookie + visiting /dashboard → redirect to /login
- Has cookie + visiting /login or /register → redirect to /dashboard

Layer 2 — Dashboard useEffect

Client-side guard as backup. Waits for `loading === false` before making any redirect decision:

```
useEffect(() => {
  // CRITICAL: only redirect after loading completes
  // Redirecting while loading=true kicks out valid users
  if (!loading && !user) {
    router.push("/login");
  }
}, [user, loading, router]);
```

8. Setup & Running

8.1 Backend Setup

1. **Install dependencies**

```
cd backend
npm install
```

2. **Create .env file** (copy from .env.example and fill in secrets)

```
PORT=5000
CLIENT_URL=http://localhost:3000
ACCESS_TOKEN_SECRET=your_very_long_random_secret_here
REFRESH_TOKEN_SECRET=another_very_long_random_secret_here
ACCESS_TOKEN_EXPIRES_IN=15m
REFRESH_TOKEN_EXPIRES_IN=7d
NODE_ENV=development
```

3. **Start the server**

```
npm start
# or for development with auto-reload:
npx nodemon server.js
```

8.2 Frontend Setup

4. **Install dependencies**

```
cd frontend
npm install
```

5. **Create .env.local**

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

6. **Start the dev server**

```
npm run dev
```

7. **Open in browser**

Visit `http://localhost:3000` — you will be redirected to `/dashboard`, then to `/login` since you have no session yet.

8.3 Testing the Flow

Step	What to do / What to expect
1. Register	Go to /register, fill form → redirected to dashboard

2. View dashboard	See your user info cards and the "Test API" button
3. Test API	Click "Test API" → calls GET /auth/me → shows your user JSON
4. Refresh page	Dashboard stays — session restored via httpOnly cookie
5. Logout	Click Logout → redirected to /login, cookie cleared
6. Back to dashboard	Visit /dashboard → middleware redirects back to /login
7. Auto-refresh	Wait 15 min or shorten ACCESS_TOKEN_EXPIRES_IN=10s to see auto-refresh in Network tab

9. Upgrading to a Real Database

When you are ready to add persistence, only two files need to change:

9.1 MongoDB (Mongoose)

8. **Install**: `npm install mongoose`
9. **Replace store.js** with a Mongoose model:

```
// models/User.js
const mongoose = require("mongoose");
const UserSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String,
});
module.exports = mongoose.model("User", UserSchema);

// models/RefreshToken.js
const TokenSchema = new mongoose.Schema({
  token: { type: String, unique: true },
  userId: String,
  expiresAt: Date,
});
module.exports = mongoose.model("RefreshToken", TokenSchema);
```

10. **Update authController.js** to use `User.findOne()`, `user.save()`, `RefreshToken.create()`, etc.

9.2 PostgreSQL (Prisma)

11. **Install**: `npm install prisma @prisma/client`
12. **Init**: `npx prisma init`
13. **Define schema** in `prisma/schema.prisma`:

```
model User {
  id        String   @id @default(cuid())
  name      String
  email     String   @unique
  password  String
  tokens    RefreshToken[]
}

model RefreshToken {
  id        String   @id @default(cuid())
  token     String   @unique
  userId    String
  user      User     @relation(fields: [userId], references: [id])
  expiresAt DateTime
}
```

No other changes needed: server.js, middleware files, routes, and the entire frontend stay exactly the same. Only store.js and authController.js need to be updated.